

■ Laporan Praktikum P7 - Abstract Class, Abstract Method, dan Interface

Nama: Akhmad Satrio Cahyo Pratama | NRP: 3125522013 | Prodi: D4 Teknik Informatika | Proyek: Sistem Bank Sampah

1. Sprint Goal (Agile Sprint P7)

Mengembangkan desain proyek P6 dengan menerapkan **abstract class** dan **interface** yang tepat sehingga struktur kelas semakin jelas, memiliki kontrak perilaku yang konsisten, mencegah instansiasi kelas umum, dan program tetap berjalan lancar tanpa merusak fitur P1–P6.

2. Sprint Backlog P7

ID	Aktivitas Sprint Backlog P7	Status
SB-01	Audit hierarchy dan behavior proyek P6	DONE
SB-02	Menentukan kandidat abstract class (Entitas)	DONE
SB-03	Menentukan abstract method (getPeran(), hitungKontribusi())	DONE
SB-04	Menentukan kandidat interface (DapatDilacak: getLocation(), getStatusOperasional())	DONE
SB-05	Memperbarui Class Diagram dengan notasi generalization & realization	DONE
SB-06	Mengimplementasikan abstract class dan interface pada seluruh subclass	DONE
SB-07	Menguji polymorphism (4 skenario), dynamic binding, dan regresi sistem	DONE

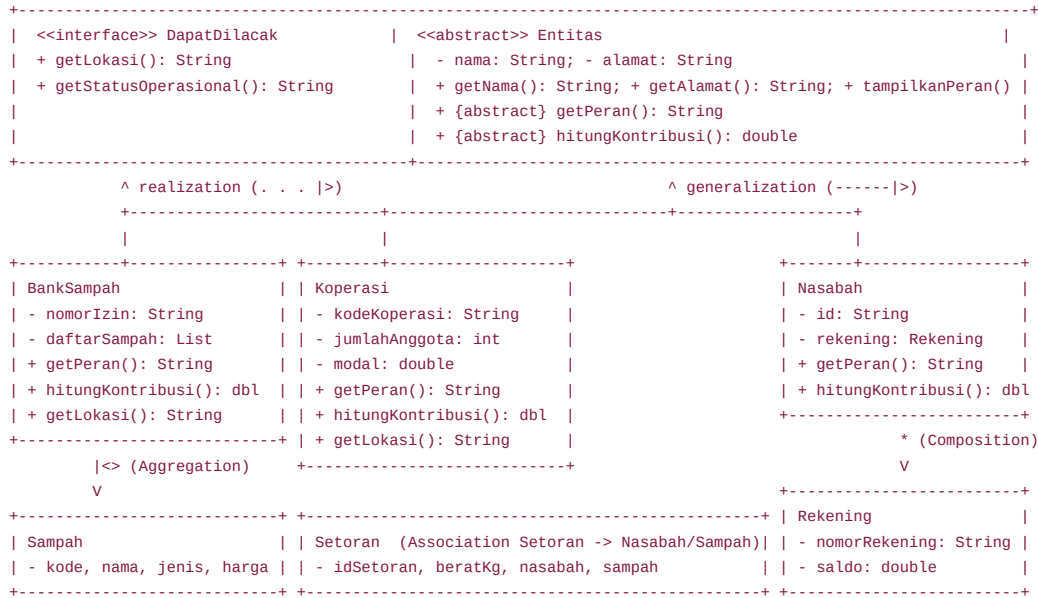
3. Tabel Audit Desain P6 ke P7 (Refactoring Analysis)

Class / Behavior	Kondisi P6	Rencana P7	Alasan Keputusan Perubahan Desain
Entitas	Class biasa (dapat di-new)	Abstract Class	Entitas hanya mewakili konsep abstrak/umum; tidak boleh diinstansiasi langsung.
getPeran()	Method biasa (return "Entitas")	Abstract Method	Setiap subclass wajib mendefinisikan identitas perannya secara spesifik.
hitungKontribusi()	Method biasa (return 0)	Abstract Method	Formula kontribusi unik per jenis entitas (saldo, omzet sampah, atau modal).
DapatDilacak	Belum ada	Interface	Mendefinisikan kontrak standar pelacakan lokasi geografis & status operasional.
BankSampah	Subclass Entitas	Implements DapatDilacak	Fasilitas fisik yang beroperasi menerima setoran dan memiliki titik lokasi.
Koperasi	Subclass Entitas	Implements DapatDilacak	Badan hukum penyedia modal yang memiliki kantor operasional terlacak.
Nasabah	Subclass Entitas	Subclass Konkret	Mengimplementasikan abstract method getPeran() dan hitungKontribusi().

Kesimpulan Audit: Refactoring P7 memperkuat integritas OOP melalui abstraction (mencegah instansiasi objek abstrak) dan interface (kontrak kemampuan melacak fasilitas tanpa terikat hierarki tunggal).

■ Halaman 2 - Arsitektur Desain, UML Class Diagram, dan Implementasi

4. Class Diagram Terbaru (P7)



5. Implementasi Abstract Class dan Abstract Method

```
public abstract class Entitas {
    private String nama, alamat;
    public Entitas(String nama, String alamat) { this.nama = nama; this.alamat = alamat; }
    public abstract String getPeran(); // Wajib di-override subclass
    public abstract double hitungKontribusi(); // Wajib di-override subclass
    public void tampilkanPeran() { // Template Method polimorfik
        System.out.println("Peran: " + getPeran() + " | Kontribusi: Rp" + formatRupiah(hitungKontribusi()));
    }
}
```

6. Implementasi Interface dan Realization pada Subclass

```
public interface DapadDilacak {
    String getLokasi();
    String getStatusOperasional();
}

public class BankSampah extends Entitas implements DapadDilacak {
    @Override public String getPeran() { return "Bank Sampah"; }
    @Override public double hitungKontribusi() { return getTotalNilaiSampah(); }
    @Override public String getLokasi() { return "PENS Kampus Sumenep [-7.0091, 113.8622]"; }
    @Override public String getStatusOperasional() { return "Buka / Menerima Setoran (Izin: " + nomorIzin + ")"; }
}
```

7. Alasan Perubahan Desain

- Mencegah Instansiasi Ilegal:** Entitas adalah abstraksi murni. Pembuatan objek langsung `new Entitas(...)` kini dicegah pada tahap compile-time.
- Menjamin Konsistensi Kontrak:** Semua turunan Entitas dipaksa compiler menyediakan method `getPeran()` dan `hitungKontribusi()`.
- Fleksibilitas Pelacakan:** Interface `DapadDilacak` memungkinkan pelacakan lokasi dan status operasional lintas kelas tanpa terikat pewarisan tunggal Java.

■ Halaman 3 - Eksekusi Program, Verifikasi Pengujian, dan Evaluasi Agile

8. Hasil Running Program (Kompilasi & Eksekusi)

```
$ javac -encoding UTF-8 *.java && java Main
=== SISTEM MANAJEMEN BANK SAMPAH - P7 ===
[SKENARIO 1] Objek Berhasil Dibuat: Nasabah (Budi), BankSampah (BS PENS), Koperasi (Kop Mandiri)
[SKENARIO 2] Abstract Method via Reference Entitas:
- refNas.getPeran() -> Nasabah | refNas.hitungKontribusi() -> Rp10.000
- refBank.getPeran() -> Bank Sampah | refBank.hitungKontribusi() -> Rp19.500
- refKop.getPeran() -> Koperasi | refKop.hitungKontribusi() -> Rp75.000.000
[SKENARIO 3] Method via Reference Interface DapatDilacak:
- BankSampah -> Lokasi: PENS Kampus Sumenep | Status: Buka / Menerima Setoran
- Koperasi -> Lokasi: Jl. Trunojoyo 3 | Status: Aktif / Permodalan Daur Ulang
[SKENARIO 4] Iterasi Entitas[]: 5 Elemen (3 Nasabah, 1 BankSampah, 1 Koperasi) berjalan dinamis!
Total Kontribusi Polimorfik: Rp75.037.000 | Regresi P4 (Komposisi/Agregasi/Asosiasi): SUKSES
```

9. Tabel Pengujian Polymorphism (4 Skenario Modul P7)

N o	Skenario Pengujian	Hasil yang Diharapkan	Hasil Aktual & Status
1	Membuat object subclass konkret	Object berhasil dibuat tanpa error	BERHASIL: Nasabah, Bank, Koperasi dibuat
2	Panggil abstract method via reference superclass	Method subclass yang sesuai dijalankan	BERHASIL: Dynamic binding getPeran/kontribusi
3	Panggil method via reference interface	Implementasi interface dieksekusi	BERHASIL: getLokasi() & status operasional
4	Beberapa object dalam array/list bertipe superclass/interface	Setiap object menjalankan perilakunya sendiri	BERHASIL: 1 loop Entitas[] & List terlacak

10. Sprint Review (Verifikasi 7 Checklist)

Item Evaluasi Sprint Review	Hasil Implementasi & Verifikasi	Status
1. Abstract class berhasil dibuat	Class Entitas dijadikan abstract, new Entitas() dicegah compiler	VERIFIED
2. Abstract method diimplementasikan subclass	getPeran() dan hitungKontribusi() dioverride oleh semua subclass	VERIFIED
3. Minimal 2 subclass konkret dapat digunakan	3 subclass aktif: Nasabah, BankSampah, Koperasi	VERIFIED
4. Interface berhasil diimplementasikan	DapatDilacak diterapkan pada BankSampah dan Koperasi	VERIFIED
5. Class diagram sesuai dengan kode	Notasi generalization & realization konsisten 100% dengan kode	VERIFIED
6. Pengujian polymorphism berhasil	4 skenario pengujian dan dynamic binding lolos pengujian	VERIFIED
7. Fitur proyek sebelumnya tetap berjalan	Regresi P4 (komposisi, agregasi, asosiasi) & P6 (overloading) utuh	VERIFIED

11. Sprint Retrospective

What Went Well? Penerapan *abstract class* dan *interface* memberikan arsitektur yang sangat terstruktur. Kontrak metode memaksa keseragaman implementasi tanpa menghilangkan fleksibilitas perilaku unik subclass.

What Went Wrong? Perlu kecermatan menentukan kontrak interface agar tidak tumpang tindih dengan tanggung jawab abstract class.

Improvement: Menyiapkan rancangan integrasi menyeluruh seluruh konsep P1–P7 untuk menghadapi Evaluasi Praktikum UTS (P8).